

Weather-Aware VEC Offloading

Phase 1 Report: System Model and Environment

Alireza Malekhosseini Student No: 401106566

Aida Jalali Student No: 401170542

Teaching Assistant: Mr. Abbas Shirang

Overview

This report explains the first phase of the weather-aware VEC offloading project. The main goal of this phase was to build the simulation environment: vehicles, tasks, weather effects, basic communication parameters, and initial evaluation outputs. The SAC-based Zone Manager is planned for the next phase, so here the focus is on preparing the environment and checking that the generated data behaves correctly under different weather conditions.

1 Project Objective

The objective of this project is to design a weather-aware adaptive task offloading system for Vehicular Edge Computing. The final system will use a SAC deep reinforcement learning agent as the Zone Manager. The agent will make offloading decisions using a state representation that includes weather, vehicle mobility, task properties, and available computing/network resources.

Phase 1 focuses on the system model and environment. Its goal is not yet to train the SAC agent, but to build a clean simulation pipeline that can generate weather-aware inputs and evaluation metrics. The implemented Phase 1 environment supports:

- VEC vehicle and task generation from mobility traces.
- Weather-specific modification of speed, task load, deadlines, signal loss, and packet loss ratio.
- Output datasets for vehicles, soft tasks, optional hard tasks, and metrics.
- A simple random offloading baseline over local, fog, and cloud execution.
- Graphs for latency, energy consumption, deadline misses, and packet losses.

2 VEC System Model

The modeled VEC architecture has three execution layers:

1. **Local vehicle layer:** A vehicle executes its own task locally.

2. **Fog/edge layer:** A nearby fog node executes an offloaded task with lower computation delay than the vehicle, but with transmission cost and packet loss risk.
3. **Cloud layer:** The cloud provides stronger computation power, but adds backhaul delay and higher communication cost.

The Phase 1 implementation represents each vehicle by its identifier, position, speed, lane, type, computing power, and active weather scenario. Each task contains its creator, release time, deadline, execution time, data size, cycles per bit, power consumption, deadline class, signal loss increase, and packet loss increase.

3 Implemented Pipeline

The implemented project is organized as a sequential generation and evaluation pipeline. If no external SUMO mobility trace is available, the project first creates a synthetic mobility file. Then it generates a weather schedule, applies weather-aware modifiers during vehicle/task generation, runs a random offloading baseline, and produces result charts. The actual output images from this pipeline are used in the next sections.

Instead of treating the simulation as only a collection of code files, the environment can be described as a set of connected components:

- **Mobility component:** provides the position, speed, lane, and type of each vehicle at each timestep.
- **Weather component:** assigns the active weather scenario and provides the related modifiers for speed, task complexity, deadline pressure, signal loss, and packet loss.
- **Task generation component:** creates computational tasks for vehicles based on traffic state, vehicle movement, and weather effects.
- **Execution model:** estimates the latency, energy, and reliability of local, fog, and cloud execution.
- **Evaluation component:** summarizes the results and produces the comparison charts required for the project.

The actual Python filenames are useful for implementation tracking, but in the report the important point is the responsibility of each component and how it supports the weather-aware VEC model.

4 Weather-Aware Environment Model

The project description defines five weather scenarios: Base, Rain, Snow, Fog, and Mixed. In the current Phase 1 implementation, the four explicit scenarios Base, Rain, Snow, and Fog are implemented. The Mixed scenario is listed as future work because it requires a combination policy for varying conditions over time or zones.

Table 1: Weather scenario effects required by the project description.

Scenario	Speed	Deadline	Cycles/Bit	Task Rate	Signal Loss	PLR
Base	V_{max}	Normal	C	R	PL	N
Rain	−15%	Tight	1.2–1.4×	+20%	+2–4 dB	+15–20%
Snow	−30%	Tightest	1.5–1.8×	−50%	+1–2 dB	+5–10%
Fog	−60%	Relaxed	1.3–1.6×	+25%	+4–6 dB	+25–30%
Mixed	Varying	Varying	Varying	Varying	Varying	Varying

Table 2: Implemented Phase 1 weather modifiers.

Scenario	Speed Factor	Deadline Class	Cycles/Bit	Task Rate	PLR Increase
BASE	1.00	Normal	1.0×	1.00×	0%
RAIN	0.85	Tight	1.2–1.4×	1.20×	15–20%
SNOW	0.70	Tightest	1.5–1.8×	0.50×	5–10%
FOG	0.40	Relaxed	1.3–1.6×	1.25×	25–30%

5 Task and Vehicle Generation

The generator reads mobility timesteps and creates one XML chunk for vehicles and one XML chunk for tasks. Vehicle speeds are multiplied by the active weather speed factor. For each timestep, tasks are generated according to traffic density, vehicle speed, and the active weather task-rate multiplier.

Each generated task includes weather-aware fields:

- `weather_scenario`
- `deadline_type`
- `cycles_per_bit`
- `path_loss_increase_db`
- `plr_increase_percent`
- `dataSize`, `exec_time`, and `deadline`

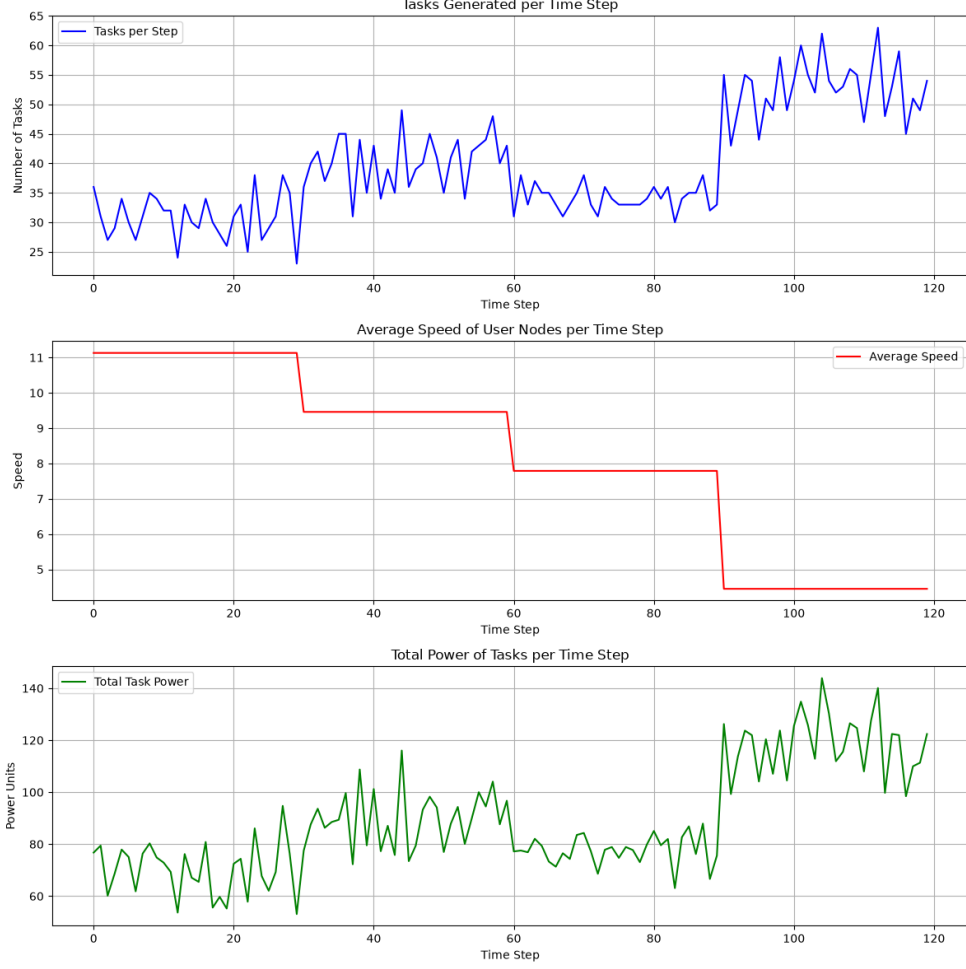


Figure 1: Generated Phase 1 metrics: task count, average vehicle speed, and total task power over simulation time.

Figure 1 shows the first validation output of the generated environment. It is not a final algorithm comparison; its purpose is to show that the simulator is producing a dynamic workload and that the weather modifiers are visible in the generated data.

The first subplot reports the number of generated tasks at each timestep. A realistic VEC environment should not create exactly the same workload at every moment, because task generation depends on vehicle state, lane density, and weather. In the implemented model, weather changes the task generation rate: Rain and Fog increase the rate, while Snow decreases it. Therefore, the task-count curve is a useful check that the workload changes over time and that the offloading algorithm in the next phase will face different levels of pressure.

The second subplot reports the average vehicle speed. This metric is directly related to the weather model because each scenario applies a speed factor to the mobility trace. Base keeps the original speed, Rain reduces it by 15%, Snow by 30%, and Fog by 60%. This matters for VEC because vehicle positions and movement affect the distance to fog nodes, communication delay, and the stability of a possible offloading decision.

The third subplot reports the total power demand of generated tasks at each timestep. This shows how much computational load enters the system over time. When task count, task size, and cycles per bit change, the required execution power also changes. This plot is useful because the future SAC Zone Manager should learn decisions under changing

workload intensity, not only under one fixed traffic condition.

6 Important Implementation Logic

This section shows the main logic used in the implementation. The snippets are shortened versions of the real code, so they focus on the algorithmic idea rather than every programming detail.

6.1 Weather Scenario Selection

At each simulation timestep, the environment reads the active weather scenario from the generated weather schedule. If the exact timestep is not found, the latest earlier scenario is used. This allows the weather to stay active over an interval of time.

Listing 1: Weather scenario lookup for each timestep.

```
def scenario_for_time(schedule, step):
    if step in schedule:
        return schedule[step]

    earlier_steps = [t for t in schedule if t <= step]
    if not earlier_steps:
        return BASE

    return schedule[max(earlier_steps)]
```

After selecting the scenario, the simulator loads the corresponding weather effect. Each effect contains speed factor, deadline type, cycles-per-bit multiplier, task-rate multiplier, signal loss increase, and packet-loss increase.

Listing 2: Weather effect structure used by the environment.

```
WeatherEffect(
    speed_factor,
    deadline_label,
    deadline_free_time_range,
    cycles_per_bit_multiplier_range,
    task_generation_rate_multiplier,
    path_loss_increase_db_range,
    plr_increase_percent_range
)
```

6.2 Applying Weather to Vehicles and Tasks

The first direct effect of weather is applied to vehicle speed. For example, Rain uses a speed factor of 0.85, Snow uses 0.70, and Fog uses 0.40. This changes vehicle mobility before tasks are generated or offloading is evaluated.

Listing 3: Weather-aware vehicle speed update.

```
vehicle_speed = original_speed * weather_effect.speed_factor
```

Task generation is also weather-aware. The number of tasks depends on traffic condition and is multiplied by the weather task-rate factor. High traffic or low speed creates more pressure on the system.

Listing 4: Simplified weather-aware task generation logic.

```
if lane_count > lane_threshold or vehicle_speed < speed_threshold:
    task_rate = 3.0 * weather_effect.task_generation_rate_multiplier
else:
    task_rate = 0.8 * weather_effect.task_generation_rate_multiplier

num_tasks = poisson_sample(task_rate)
```

For each generated task, weather modifies both computation and communication parameters. The task deadline is created from execution time plus a scenario-dependent free-time range. The packet-loss increase and path-loss increase are saved inside the task record for later evaluation.

Listing 5: Main task parameter generation logic.

```
cycles_per_bit = random(base_min, base_max) \
    * weather_effect.sample_cycles_multiplier()

data_size = random(data_min, data_max)
exec_time = (data_size * cycles_per_bit) / vehicle_frequency

deadline_free = random(weather_effect.deadline_free_time_range)
deadline = current_step + exec_time + deadline_free

task.weather_scenario = weather_effect.scenario
task.deadline_type = weather_effect.deadline_label
task.path_loss_increase_db = weather_effect.sample_path_loss_increase_db()
task.plr_increase_percent = weather_effect.sample_plr_increase_percent()
```

6.3 Baseline Execution Logic

The random baseline is used only to check the evaluation pipeline in Phase 1. For each task, one execution target is selected randomly from local vehicle, fog node, and cloud.

Listing 6: Random offloading decision.

```
target = random_choice([LOCAL, FOG, CLOUD])
```

For local execution, there is no communication delay or packet loss. For fog execution, transmission delay depends on task size, fog bandwidth, and distance from the nearest fog node. For cloud execution, the model adds a fixed backhaul delay. Weather packet-loss increase is added to the base packet loss of fog and cloud.

Listing 7: Simplified latency and packet-loss model.

```
if target == LOCAL:
    latency = task.exec_time
    packet_loss_percent = 0

elif target == FOG:
```

```

distance_delay = distance(vehicle, nearest_fog) * fog_delay_factor
transmission_delay = task.data_size / fog_bandwidth + distance_delay
latency = transmission_delay + task.exec_time / fog_speedup
packet_loss_percent = fog_base_plr + task.plr_increase_percent

else: # CLOUD
    transmission_delay = task.data_size / cloud_bandwidth + backhaul_delay
    latency = transmission_delay + task.exec_time / cloud_speedup
    packet_loss_percent = cloud_base_plr + task.plr_increase_percent

```

Finally, each task is marked as successful or failed. A task fails if the packet is lost or if its finish time passes the deadline. This is why the result charts include both packet loss and deadline miss counts.

Listing 8: Deadline miss and packet loss decision.

```

packet_lost = random_number() < packet_loss_percent / 100
finish_time = task.release_time + latency
deadline_missed = packet_lost or finish_time > task.deadline

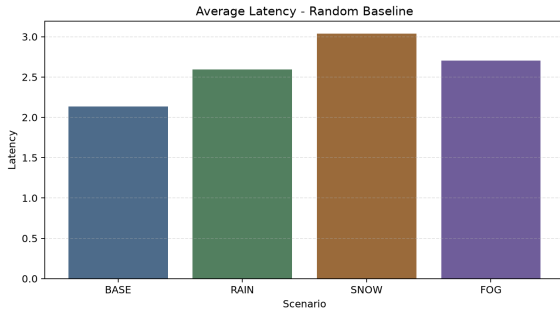
```

7 Phase 1 Evaluation Results

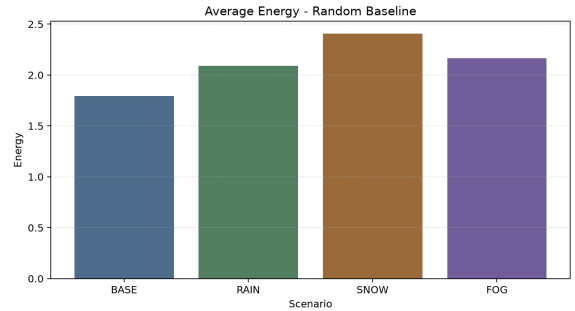
Table 3 summarizes the generated random-baseline results by weather scenario. The baseline is included only as an initial evaluation method for Phase 1. The numbers are not meant to prove that random offloading is a good method; they are mainly used to check that the environment produces different behavior in different weather conditions and that the evaluation code can calculate all required metrics.

Table 3: Random baseline summary over generated Phase 1 tasks.

Scenario	Tasks	Deadline Misses	Packet Losses	Avg. Latency	Avg. Energy
BASE	922	16	16	2.137	1.794
RAIN	1210	164	164	2.596	2.092
SNOW	1022	86	86	3.039	2.407
FOG	1584	308	308	2.706	2.164



(a) Average latency



(b) Average energy

Figure 2: Average latency and energy consumption for the random baseline.

Figure 2 compares average latency and average energy consumption for the random baseline. The Base scenario has the lowest average latency, about 2.137, because it has no weather penalty on speed, cycles per bit, signal loss, or packet loss. Rain increases the average latency to about 2.596 because tasks become heavier and packet loss becomes more likely. Snow gives the highest average latency, about 3.039, even though its task generation rate is lower. This happens because Snow applies the strongest cycles-per-bit multiplier, so each generated task becomes computationally heavier. Fog has average latency around 2.706, which is also high because of severe communication degradation and increased task generation.

The energy chart follows a similar pattern. Base has the lowest average energy consumption, about 1.794. Rain increases it to about 2.092, Fog to about 2.164, and Snow reaches the highest value, about 2.407. This result makes sense for the implemented model because higher execution load and longer transmission/execution time increase energy consumption. Snow is especially expensive because its tasks require more computation per bit.

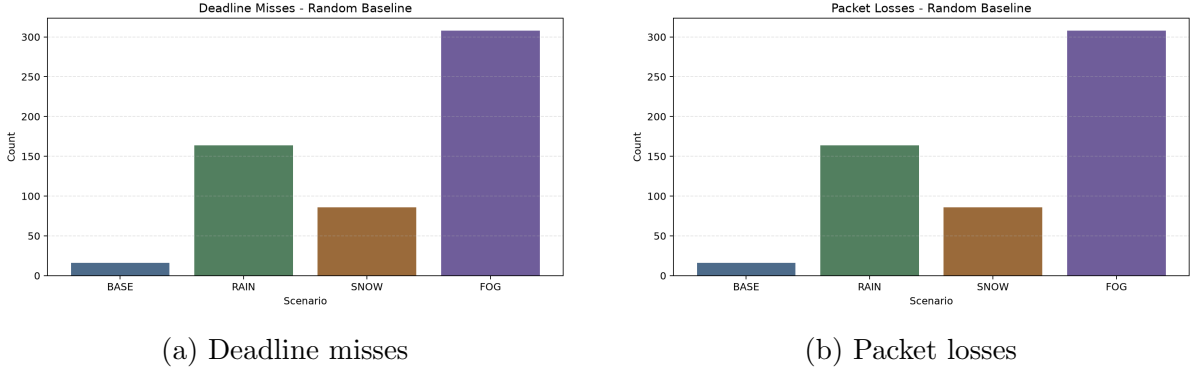


Figure 3: Reliability-related outputs for the random baseline.

Figure 3 focuses on reliability. In this simulation, packet loss directly affects deadline success, so the number of packet losses and deadline misses are equal for each scenario. Base has only 16 packet losses and deadline misses, which is expected because it has no extra weather packet-loss penalty. Rain increases both values to 164 because it adds a 15–20% PLR increase. Snow has 86 packet losses and deadline misses. Although Snow creates heavier tasks, its PLR increase is smaller than Rain and Fog, and its task generation rate is lower. Fog produces the worst reliability result with 308 packet losses and deadline misses. This is consistent with the Fog model because it combines the highest PLR increase, 25–30%, with a higher task generation rate.

Overall, the result figures show that the implemented environment reacts to weather in the expected direction. Bad weather increases computation cost, communication risk, or both. This confirms that the generated Phase 1 environment is suitable for later comparison between a weather-aware SAC Zone Manager and non-weather-aware baselines.

8 What Was Implemented

In this phase, the implementation covers the main environment-building requirements:

- Weather-aware vehicle state generation.

- Weather-aware task generation.
- Scenario-based speed, deadline, cycles-per-bit, task-rate, signal-loss, and PLR modifiers.
- Generated XML datasets for vehicles and tasks.
- Generated CSV metrics and visualization.
- Baseline execution path for local, fog, and cloud offloading.
- Result charts for latency, energy, deadline misses, and packet losses.

9 Limitations and Phase 2 Work

The current implementation is intentionally scoped to Phase 1. The main limitations are:

- The Mixed weather scenario is not fully implemented yet.
- The default mobility trace is synthetic and should later be replaced by a real SUMO mobility trace.
- The communication model uses simple packet-loss and delay assumptions, not a detailed physical wireless channel model.
- The current baseline is random; stronger non-weather-aware baselines should be added in Phase 2.
- The SAC-based Zone Manager has not been implemented yet.

In Phase 2, the environment can be used as the foundation for training and evaluating the SAC Zone Manager. The SAC state space should include weather scenario, vehicle mobility, task deadline/load, edge/cloud resource availability, and communication reliability. The reward function should combine latency, energy consumption, deadline satisfaction, and packet delivery reliability.

10 Short Summary

This phase establishes the system model and simulation environment required for weather-aware VEC offloading. The implemented pipeline generates weather-modified mobility, tasks, communication degradation parameters, and evaluation outputs. The random baseline confirms that the environment can produce the expected comparison metrics for latency, energy consumption, deadline misses, and packet loss. This gives a practical base for the next phase, where SAC and stronger baseline algorithms can be implemented and evaluated across all weather scenarios.